OnboardAI Project Guide

This guide explains the whole project twice. Part A uses ordinary language and assumes no background in computing. Part B covers the same ground with the technical detail a reviewer or examiner would expect.

Read Part A if you want to understand what we built and why. Read Part B if you need to know how.

Part A: The Project in Plain Language

A.1 What problem are we solving

When a company hires somebody, a surprising amount of paperwork follows. The new employee has to send in a copy of their passport or identity card, a signed offer letter, bank details so they can be paid, and depending on the job, things like a safety certificate or a professional licence. Somebody in the human resources team has to look at each of these and decide whether it is acceptable.

At the same time the new employee has questions. How much holiday do I get. When does my training start. Who do I talk to about my laptop. These are not difficult questions, but they have to be answered, and the answers differ from company to company.

All of this is done by hand in most organisations. It is slow, it is repetitive, and it is easy for somebody to fall through the gap. A new employee who is waiting for an answer for a week forms a poor opinion of the company, and that opinion is hard to repair later. Replacing somebody who leaves in their first year costs the company several months of that person's salary, so this is an expensive problem rather than a small annoyance.

A.2 What our system does

We are building a system that helps with three specific parts of this.

The first is checking documents. When somebody uploads a certificate, the system reads it, works out what kind of document it is, pulls out the important dates, and decides whether it is acceptable. If a safety certificate expired two years ago, it says so and explains why it was rejected. If everything looks fine it approves it. If it is not sure, it passes the document to a human rather than guessing.

The second is answering questions. The system searches the company's own policy documents and answers from what it finds, quoting which document the answer came from. If the handbook does not cover the question, it says it does not know and offers to pass the question to a human. It is specifically not allowed to make up an answer, because a confident wrong answer about holiday or pay causes real problems for somebody.

The third is spotting people who are falling behind. The system looks at how much progress each new employee has made, how long since they last did anything, how many of their documents were rejected, and how their messages sound, and produces an estimate of how likely they are to get stuck. It also explains why, so instead of saying "this person is at 48 per cent risk" it says "48 per cent, mainly because no tasks are complete and one document was rejected".

A.3 Where we are now

The project is split into two halves. The first half, which is what the mid term review covers, is about the data. The second half is about building and testing the models that learn from it.

We have finished the first half. That means we have built our datasets, examined them properly, cleaned and prepared them, split them into a portion for teaching the system and a portion for testing it, and written automated checks that confirm all of this is correct.

We have also built a fair amount of the second half already, including a working website, but that is not what we are being assessed on right now and we are careful not to present it as if it were.

A.4 Where our data came from, and why that matters

This is the part we expect to be asked about, so it is worth being direct.

We generated our own data rather than using real data. There are two reasons.

Real onboarding documents are people's passports and bank details. We cannot legally collect those from anyone, and there is no public collection of Indian onboarding paperwork we could use instead.

Real information about who struggled during onboarding does not exist for us, because our system has never been used by a real company. There is nothing to learn from yet.

So we wrote programs that produce realistic fake documents, messages and employee histories. The important thing is that we tried to make the fake data hard rather than easy. For the documents we simulate bad scans, with the top of the page missing, text smudged, and sentences from a different document accidentally included. For the messages we include genuinely vague ones like "any update?" where it is impossible to tell what the person wanted. For the employee histories we include a hidden factor that affects the outcome but that the system cannot see, so perfect prediction is impossible on purpose.

The limitation is real and we state it everywhere. Our results tell us how well our system agrees with our own data generator. They do not tell us how it would perform on a real passport. Replacing the fake documents with a real public collection is the first thing on our list for the next phase.

A.5 The three things we found that surprised us

Our first model was too good, and that was a problem. The first version of our document classifier got every single answer right. We were pleased for about ten minutes, then realised that a perfect score with no variation means the task was too easy rather than the model being clever. The fake documents were too different from each other. We rebuilt them with damage and overlap until the hardest category scored about 94 per cent, which is a number we actually believe.

We were wrong about our own data. We discovered that some messages appeared in both the teaching set and the testing set, which normally makes results look better than they are because the system has already seen the answer. We assumed this was inflating our scores. Then we measured it, and found the opposite. The system actually did slightly worse on those repeated messages. The reason is that the messages that get repeated are the short vague ones like "thank you", which are the hardest to interpret anyway. The problem is still a problem, but our explanation for it was wrong, and we have written that down rather than quietly correcting it.

Three of our measurements were secretly the same measurement. When we looked at how our employee features relate to each other, three of them, the stage someone is at, their progress percentage, and the fraction of their tasks completed, turned out to be almost identical. They move together almost perfectly. This explains something that had confused us, which is why removing any one of them barely changed anything. Each one individually seemed useful, but between them they only carried one piece of information.

A.6 Who did what

Four of us worked on this.

Kritika Sah led the team, which meant defining what problem we were actually solving, setting the objectives, deciding the scope and keeping everybody pointed in the same direction.

Rohith K Nair wrote the code, which covers the data generators, the preprocessing, the splitting and the analysis.

Prisha Saini tested it, which means writing checks that try to break the system and prove the data is what we say it is. She wrote 34 test cases covering normal use, edge cases like empty files, and deliberately malicious input.

Pasham Srikar handled quality assurance, which is a different job from testing. Testing asks whether the system works. Quality assurance asks whether our way of working can be trusted, and whether the claims we are making are actually supported by evidence. He keeps the register of known problems and checks

that every number we quote can be traced back to the code that produced it.

Part B: The Project in Technical Terms

B.1 System overview

OnboardAI is a full stack application with a machine learning and natural language processing layer wired into the request path rather than attached as a separate demonstration.

The stack is React 19 with Vite and Tailwind on the front end, and FastAPI with SQLAlchemy over SQLite on the back end. The analytical layer is scikit-learn and NumPy. Language model inference runs locally through Ollama, with Google Gemini and Groq available as alternatives behind a provider abstraction. No paid service is required anywhere in the project.

Three prediction problems are addressed, each with its own dataset:

Problem	Data type	Target	Samples
Document type verification	Unstructured text	9 classes	4,950
Intent and sentiment classification	Unstructured text	8 and 3 classes	3,360
Onboarding stall prediction	Structured tabular, 12 features	Binary	6,000

B.2 Data generation methodology

All three datasets are synthetic. The generators were designed around one principle, which is that the generated task must be hard enough for the resulting metric to mean something.

Document corpus. Twenty one templates across nine classes, with slot randomisation over names, dates, identifiers and organisations. Degradation is applied in three tiers. The severe tier strips the header, reduces the document to a fragment, injects ten to twenty per cent character level noise using OCR confusion pairs, and appends sentences drawn from other classes. The tier mix is approximately 40, 38 and 22 per cent.

The first version of this generator had no degradation and produced a classifier scoring 1.000 with zero cross validation variance. That was treated as a broken benchmark rather than a result, and the generator was rebuilt.

Message corpus. Forty five templates across eight intents, carrying two independent labels. Irreducible error is introduced deliberately through a pool of uninformative messages whose label cannot be recovered from the text, and through blended messages that span two intents.

A labelling fault was found and corrected during development. Both the positive and neutral prefix pools contained an empty string, so a message could be labelled positive while carrying no positive signal, which is mislabelled training data.

Journey dataset. A latent variable process. Three traits are drawn per simulated employee, `engagement`, `friction` and `support`, each standard normal. Observable features are derived as noisy functions of these traits, for example

```
days_since_last_activity ~ Exponential(exp(0.75 - 0.65 * engagement + 0.30 * friction))
avg_sentiment            = tanh(0.55 * engagement - 0.85 * friction + 0.45 * support + e)
```

The outcome is a Bernoulli draw on a logit combining the traits, two direct blocking effects for rejected documents and inactivity, and an explicitly unobserved normal term. The unobserved term places a ceiling on achievable performance by construction.

B.3 Preprocessing

Text. Lowercasing and accent stripping, then a feature union of word level TF-IDF over one and two grams and character level TF-IDF over three to five grams within word boundaries. Sublinear term frequency is applied. Minimum document frequency is two.

The character n-grams are the deliberate choice. Word features are destroyed by the character noise in the severe tier, while character n-grams degrade gracefully.

Tabular. Standard scaling to zero mean and unit variance, fitted inside a scikit-learn `Pipeline` so that it refits on each training fold during cross validation and never observes the test fold.

Not performed. Missing value imputation, because all three datasets contain zero missing values, verified by test TC-02. Outlier removal, although the interquartile range rule identifies a substantial number, because the affected features are counts and waiting times whose right tail is the target behaviour.

Evidence of the vectorisation step on the document corpus:

Measure	Before	After
Representation	4,950 raw strings	Sparse matrix, 4,950 rows
Features at `min_df=1`	41,000 approximately	Pruned
Features at `min_df=2`	n/a	11,482 word n-gram features
Density	Dense text	Over 99 per cent sparse

B.4 Exploratory analysis findings

Ten figures were produced before any model code was written. The four findings that changed our approach:

1. Document and intent classes are balanced by construction, with a balance ratio of 1.00, so

plain accuracy is a defensible headline metric for those two problems. Sentiment is imbalanced by design, so balanced class weights are used.

1. The journey feature correlation matrix shows `stage`, `progress` and `pct_tasks_complete`

correlating above 0.90 with one another, reaching 0.97 for the strongest pair. This multicollinearity explains the small magnitude of single feature ablation effects.

1. No individual feature correlates with the target above 0.45 in absolute value. The strongest

is `avg_sentiment` at approximately negative 0.45, followed by `docs_pending` at positive 0.39. This is consistent with a generator that withholds part of the causal structure.

1. Sentiment separates the outcome classes in the expected direction but with substantial

distributional overlap, indicating a contributing rather than a decisive signal.

B.5 Splitting and leakage audit

All datasets use a 75 and 25 stratified split with `random_state=42`. Stratification holds class proportions to within 0.0005 of the full dataset.

Leakage is controlled structurally. The vectoriser and the scaler sit inside the pipeline object, so cross validation refits them per fold.

The audit found a defect. The document corpus has zero fold overlap. The message corpus has 200 exact duplicates, of which 70 appear in both folds, representing 8.3 per cent of that test set.

The hypothesis that this inflated our scores was tested rather than assumed. The sentiment classifier was trained once and the test set scored in two partitions:

Partition	Rows	Accuracy
Appears in training (leaked)	76	0.9868
Does not appear in training	764	0.9974

The leaked partition scores lower, so the hypothesis is rejected. The duplicated strings are the short ambiguous ones, which are both the most likely to be generated twice and the hardest to classify. Duplication is concentrated in the difficult cases.

The leakage remains a defect. The correct remedy is deduplication before splitting, or grouped splitting. It has been documented rather than silently corrected, and two tests assert its presence and bound its size.

B.6 Validation

Thirty four test cases expanding to fifty five checks, all currently passing, executing in approximately twenty seconds with no network access.

Coverage spans data validation, preprocessing verification, split integrity, functional contract, boundary input, exceptional input, access control, regression, performance and security.

Two design decisions constrain the suite. No test invokes a language model, because a result dependent on sampling temperature cannot serve as a regression gate. All file handling tests redirect storage to a temporary directory.

Known gaps: no API level tests, no authentication tests since no authentication exists, no frontend tests, no answer quality evaluation, and no automatic metric threshold enforcement.

B.7 Components built ahead of schedule

These are outside the mid term assessment boundary and are summarised only.

Document pipeline. Extract, classify, extract fields, validate, route. Routing is confidence gated: a blocking check rejects, confidence below 0.60 or any warning routes to a human, and only a confident classification with clean checks approves automatically.

Retrieval. Hybrid TF-IDF and LSA scoring over a policy corpus, access controlled by organisation and department. A measured example of the semantic component doing real work: the query "can I take time off before my probation ends" scores 0.339 lexically and 0.846 semantically against the relevant section.

Agents. Six specialists with tool sets enforced by construction rather than by instruction. Routing uses the trained intent classifier rather than a language model, which is cheaper, deterministic and measurable. Tools are bound in a closure over the authenticated employee identifier, so no prompt can cause access to another record.

Risk scoring. Logistic regression over 12 features with exact per feature contributions to the log odds. For a standardised linear model these are the SHAP values, so attribution is deterministic.

B.8 Defects found and their resolution

Ref	Defect	Severity	Resolution
1	Date role assignment used whole line keyword search, marking a valid passport expired	Blocking	Nearest preceding label logic, regression test TC-18

Ref	Defect	Severity	Resolution
2	Certificate number pattern matched the prefix of the word CERTIFICATE	Major	Required qualifier token, regression test TC-19
3	Two risk features hardcoded at inference while trained on varying values	Blocking	Both features removed, model retrained
4	Sentiment labelling permitted positive samples with no positive marker	Major	Marker guaranteed unless intent carries tone implicitly
5	Message duplicates leak across the split	Major	Documented and bounded by tests, fix deferred
6	Three journey features correlate above 0.90	Minor	Documented for next phase

B.9 Limitations

Stated in order of how much they constrain our conclusions.

1. All data is synthetic. Every reported figure measures consistency with our own generators.
2. Sentiment accuracy is a template separability ceiling and must not be presented as model

capability.

1. Generated documents average under 400 characters, considerably shorter and structurally

simpler than real scanned certificates.

1. The message corpus contains duplicates that straddle the split.
2. The feature set contains redundancy through correlated progress measures.
3. The wider application has no authentication, and the project is not under version control.

B.10 Next phase plan

Model implementation, training, testing, evaluation and comparison, in that order.

Planned comparisons are linear support vector classification against logistic regression for document type, logistic regression as a baseline for intent and sentiment, and logistic regression against gradient boosting for stall risk.

Evaluation will report accuracy and macro averaged F1 for classification, and ROC AUC with a calibration assessment for the risk problem, since a probability used to rank people needs to be meaningful and not merely correctly ordered. Five fold cross validation and ablation results will be reported rather than a single held out figure.